🚀 Lecture 04.12: Set Methods & Algebraic Set Operations

📌 Executive Architectural Summary

Python sets provide direct operator overloads and methods corresponding to classical mathematical set theory: Union ($\cup$), Intersection ($\cap$), Difference ($\setminus$), and Symmetric Difference (Δ).

🔬 Core Engineering Mechanics

1. Mathematical Set Operations

Operation	Operator	Method	Formal Definition
Union	`A | B`	`A.union(B)`	$\{x \mid x \in A \lor x \in B\}$
Intersection	`A & B`	`A.intersection(B)`	$\{x \mid x \in A \land x \in B\}$

| **Difference** | `A - B` | `A.difference(B)` | $x \mid x \in A \land xotinB$ | | **Symmetric Diff** | `A ^ B` | `A.symmetric_difference(B)` | $x \mid (x \in A \land xotinB) \lor (x \in B \land xotinA)$ |

2. Disjoint Sets Verification

Two sets A and B are disjoint if and only if their intersection is empty:

$$A \cap B = \emptyset \iff extA.isdisjoint(B)extisTrue$$

💻 Production Python Implementation

```
set_a = {1, 2, 3, 4, 5}
set_b = {4, 5, 6, 7, 8}

print("Set A:              ", set_a)
print("Set B:              ", set_b)
```

```
print("Union (A | B):      ", set_a | set_b)
print("Intersection (A & B):", set_a & set_b)
print("Difference (A - B):  ", set_a - set_b)
print("Symm Diff (A ^ B):   ", set_a ^ set_b)

# Disjoint Testing
s1 = {1, 2, 3}
s2 = {5, 6, 7}
print(f"Are s1 and s2 disjoint? {s1.isdisjoint(s2)}")
```